PROYECTO INTEGRAL DE PORTAFOLIO · ANALYTICS ENGINEERING & BUSINESS INTELLIGENCE

Retail & E-Commerce Analytics End-to-End

Desde la Ingesta y Modelado Dimensional SQL con **dbt Core + Google BigQuery** hasta el Esquema en Estrella, Catálogo **DAX**, **Power BI DevOps (PBIP · Git/GitHub · CI/CD · Fabric Git Integration)** y Dashboard Ejecutivo.

Google BigQuery | dbt Core v1.12.5 | SQL Estándar | 28/28 Data Quality Tests | Power BI · Star Schema · DAX | PBIP · TMDL · PBIR | Git / GitHub Actions CI | Fabric Git Integration v2.0

Autor: Rubén Barrios | *Analista de Datos · Business Intelligence*

🌐 Ver Dashboard Interactivo en Power BI Web 💻 Repositorio en GitHub

01 Contexto de Negocio, Objetivos y Arquitectura End-to-End

En operaciones modernas de **Retail & E-Commerce**, los datos transaccionales suelen originarse en múltiples tablas crudas con formatos heterogéneos, descuentos sin conciliar y estados operativos mezclados (*completados, enviados, en proceso, devueltos y cancelados*). Conectarse directamente desde un reporte a tablas sin curar provoca discrepancias financieras entre áreas, métricas lentas y diagnósticos comerciales erróneos.

Este proyecto resuelve el ciclo analítico completo de punta a punta aplicando las mejores prácticas actuales de la industria: **transformación modular en la nube (`ELT`)**, **control de calidad automatizado**, **modelado dimensional en estrella**, **desarrollo BI como código (`BI-as-Code`)** y **visualización ejecutiva orientada a la toma de decisiones**.

🎯 1. Rendimiento Financiero Conciliado

Determinar con exactitud contable la Venta Neta efectiva, el Costo de Ventas (`COGS`), la Utilidad Neta y el % Margen Comercial mensual descontando promociones.

👑 2. Valor y Concentración de Clientes

Cuantificar el peso real del segmento *VIP* frente a clientes *Frecuentes* y *Activos* mediante métricas de *Customer Lifetime Value (LTV)* y Ticket Promedio.

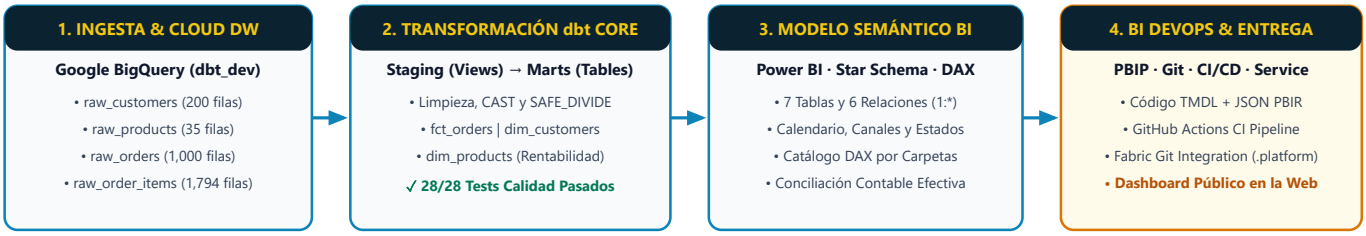
📦 3. Rentabilidad del Catálogo

Identificar qué líneas de producto (*Electrónica, Muebles, Indumentaria, Accesorios*) traccionan la facturación y cuáles aportan mayor margen neto.

🌎 4. Salud Operativa y Mercados Regionales

Medir el impacto económico de devoluciones y cancelaciones frente a los pedidos efectivos y evaluar el desempeño en los 9 países donde opera la compañía.

Arquitectura Integral del Flujo de Datos (4 Capas)



02 Fase 1: Base de Datos Transaccional e Ingesta en Google BigQuery

El punto de partida del proyecto consiste en un ecosistema transaccional reproducible de comercio electrónico (libre de datos personales sensibles, utilizando dominios sintéticos `@example.com`), estructurado en cuatro archivos semilla (`seeds/`) cargados hacia el almacén en la nube **Google BigQuery** (proyecto `dbt-retail-analytics`, dataset `dbt_dev`) mediante el comando `dbt seed`:

Tabla Cruda (Seed)	Volumen	Granularidad	Campos Principales e Información Contenida
`raw_customers.csv`	**200 filas**	1 fila por cliente	`customer_id`, nombre, apellido, email, ciudad, país (9 mercados en LATAM y España) y `signup_date`.
`raw_products.csv`	**35 filas**	1 fila por SKU	`product_id`, nombre del producto, `category` (4 líneas comerciales), `cost_price` y `list_price` tipados como `NUMERIC`.
`raw_orders.csv`	**1,000 filas**	1 fila por orden	`order_id`, `customer_id`, `order_date` (2024–2026), `channel` (*web, mobile_app, store*) y `status` operativo.
`raw_order_items.csv`	**1,794 filas**	1 fila por ítem/orden	`order_item_id`, `order_id`, `product_id`, `quantity`, `unit_price` y descuento promocional (`discount_amount`).

> **Decisión de Ingeniería en `dbt_project.yml`:** Se forzó el tipado explícito `+column_types: numeric` en precios unitarios, costos y descuentos desde la ingesta para evitar errores de redondeo de punto flotante en cálculos financieros posteriores, materializando la capa `staging` como vistas ligeras (`view`) y la capa `marts` como tablas físicas optimizadas para lectura en BI (`table`).

03 Fase 2: Modelado Dimensional SQL con dbt Core (Staging → Marts)

Siguiendo el paradigma moderno **ELT (Extract, Load, Transform)**, toda la lógica de limpieza, tipado y reglas de negocio se programó en SQL modular dentro de **dbt Core** usando referencias dinámicas `{{ ref('...') }}` que construyen automáticamente el grafo de linaje (DAG) en Google BigQuery:

3.1 Capa Staging (`models/staging/` — Materialización: `VIEW`)

- **`stg_customers.sql`:** Limpieza de espacios (`TRIM`), normalización de correos en minúsculas (`LOWER`), construcción de `full_name` y conversión explícita de `customer_id` (`INT64`) y `signup_date` (`DATE`).
- **`stg_products.sql`:** Estandarización de categorías y cálculo preventivo del margen bruto unitario (`unit_margin`) y porcentual (`margin_percentage`) empleando `SAFE_DIVIDE` para evitar fallos por división entre cero.
- **`stg_orders.sql`:** Normalización de estados operativos y canales de venta en minúsculas junto al casteo estricto de fechas transaccionales.
- **`stg_order_items.sql`:** Cálculo exacto por línea de pedido del importe bruto (`quantity * unit_price`) y del importe neto cobrado tras promociones (`item_net_amount`).

3.2 Capa Marts Analíticos (`models/marts/` — Materialización: `TABLE`)

- **`fct_orders.sql` (Tabla de Hechos de Cabecera de Órdenes):** Agrega las 1,794 líneas de detalle a nivel de las 1,000 órdenes únicas, incorporando atributos temporales (`order_year`, `order_month`, `order_year_month`), métricas financieras (`gross_amount`, `total_discount_amount`, `net_amount`) y el indicador booleano de conciliación comercial `is_completed_order`.
- **`dim_customers.sql` (Dimensión 360 del Cliente):** Consolida primera y última compra, órdenes efectivas, gasto histórico acumulado (`lifetime_spend`), ticket promedio (`average_order_value`) y clasificación automática en segmentos: **VIP** (≥ `$600`), **Frequent** (≥ `3` pedidos efectivos) y **Active**.
- **`dim_products.sql` (Catálogo Analítico de Rentabilidad por SKU):** Cruza el catálogo maestro con las transacciones efectivas para exponer unidades desplazadas, facturación neta, descuentos absorbidos y ganancia estimada (`total_estimated_profit`).

```
-- Extracto clave de conciliación comercial en models/marts/fct_orders.sql y dim_customers.sql
case when o.status in ('completed', 'processing', 'shipped') then true else false end as is_completed_order,
-- Regla de segmentación 360 alineada con las órdenes comercialmente efectivas:
case
    when coalesce(cos.lifetime_spend, 0) >= 600 then 'VIP'
    when coalesce(cos.completed_orders, 0) >= 3 then 'Frequent'
    else 'Active'
end as customer_segment
```

> **Hallazgo y Conciliación Analítica (Resolución de Caso Borde):** Durante la auditoría cruzada entre BigQuery y Power BI se detectó que evaluar segmentos únicamente sobre órdenes cerradas dejaba a clientes con pedidos activos en curso (ej. cliente `#148` con orden en estado *processing* por `$382.00`) etiquetados erróneamente como *Prospectos con $0 mil*. Al unificar el criterio comercial efectivo (`completed`, `processing`, `shipped`) tanto en SQL (`dbt`) como en `DAX`, se alcanzó consistencia contable del 100% (`VIP: 97`, `Frecuente: 72`, `Activo: 31`).

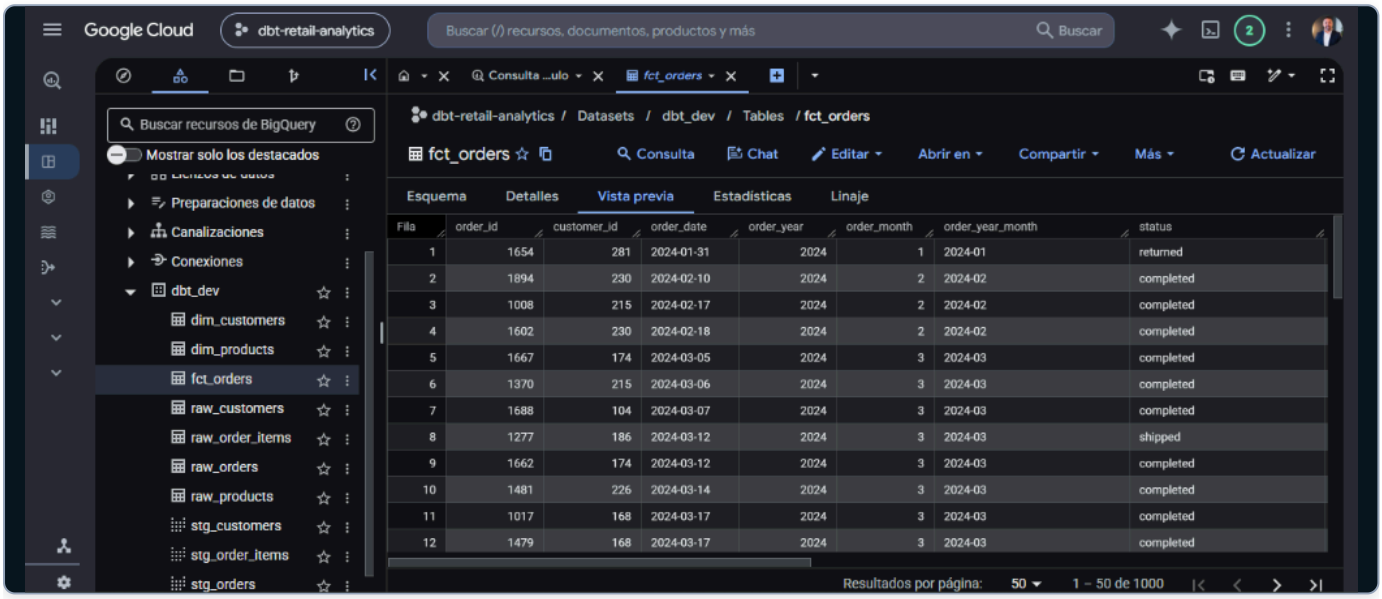


Figura 1. Almacén de datos en Google BigQuery (dataset `dbt_dev`) mostrando las tablas modeladas por dbt y la vista previa de `fct_orders`.

04 Fase 3: Gobierno y Calidad de Datos Automatizada (28/28 dbt test)

Antes de habilitar el consumo desde Power BI, se configuró una batería de **28 pruebas automáticas declarativas** en `models/staging/schema.yml` y `models/marts/schema.yml` ejecutadas en BigQuery mediante `dbt test`, complementadas con consultas de perfilado estadístico en `analyses/profile_dim_customers.sql`:

Categoría de Prueba	Regla dbt	Cant.	Validación Ejecutada en Google BigQuery	Estado
Unicidad de Claves	`unique`	7	Cero duplicados en claves primarias de `stg_*`, `fct_orders`, `dim_customers` y `dim_products`.	**PASS** ✓
Completitud Crítica	`not_null`	14	Ausencia total de nulos en IDs, emails, categorías, importes netos y llaves foráneas.	**PASS** ✓
Integridad Referencial	`relationships`	5	Toda orden e ítem apunta a un cliente, cabecera de pedido y producto válido existente.	**PASS** ✓
Dominio de Negocio	`accepted_values`	2	Estados operativos válidos (5 valores) y segmentos comerciales permitidos sin anomalías.	**PASS** ✓

05 Fase 4: Modelo Semántico en Estrella y Catálogo DAX en Power BI

Con las tablas certificadas en BigQuery, se conectó **Power BI Desktop** mediante el conector nativo de Google Cloud y se diseñó un **Esquema en Estrella de 7 tablas** con filtrado estricto unidireccional (`1:*`), separando limpiamente tablas

transaccionales de dimensiones descriptivas y tablas calculadas en DAX:

- **Tablas Transaccionales (Hechos y Detalle):** `fct_orders` (cabecera de pedidos y contenedor central de medidas) y `stg_order_items` (granularidad a nivel producto/descuento/costo para exactitud de margen por categoría).
- **Dimensiones Maestras desde BigQuery:** `dim_customers` (con columnas semánticas en español `País` y `Segmento` ordenadas jerárquicamente por `Orden Segmento`) y `dim_products` (con `Categoría` traducida al español).
- **Dimensiones Analíticas DAX:** `Calendario` (inteligencia temporal continua 2024–2026 con año, mes y orden cronológico), `Canales` (*Web, App Móvil, Tienda Física*) y `Estados` (*Completado, Enviado, En Proceso, Devuelto, Cancelado*).

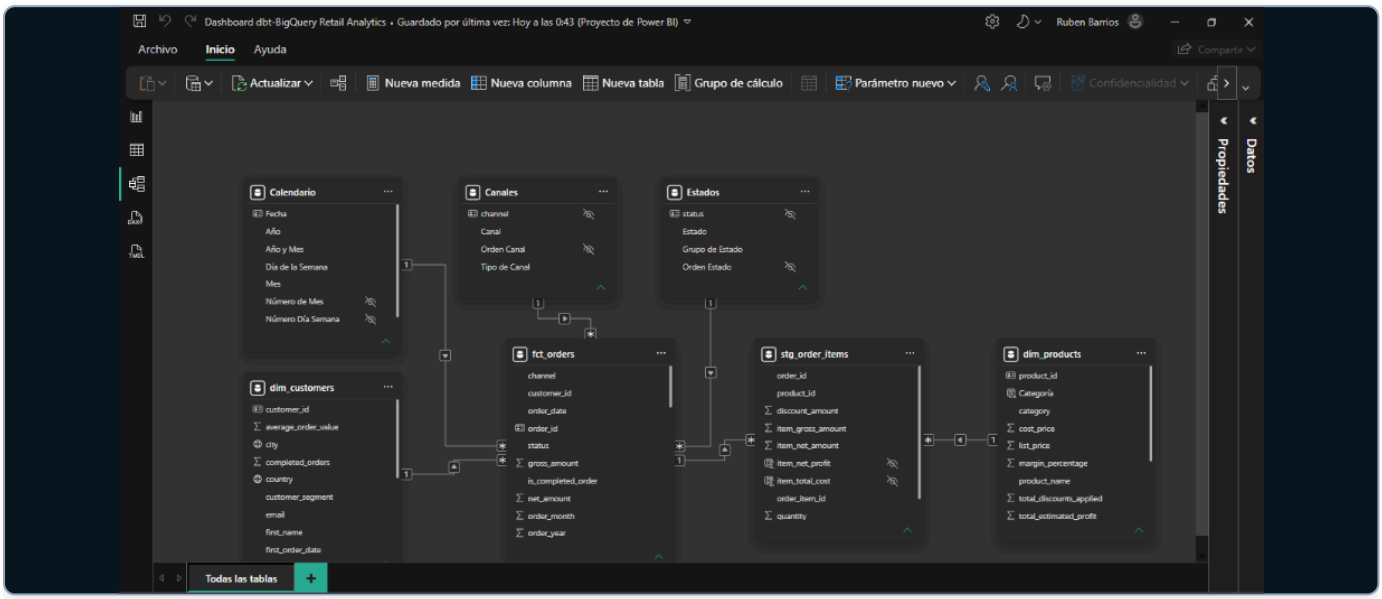


Figura 2. Vista del Modelo Relacional en Estrella en Power BI Desktop (7 tablas y 6 relaciones activas `1:*`).

Catálogo Centralizado de Medidas DAX (Organizado en Display Folders)

Se construyó un diccionario de medidas explícitas en `fct_orders.tmdl` estructurado en 4 carpetas ejecutivas (`01 KPIs Principales`, `02 Control Operativo y Descuentos`, `03 Clientes y Segmentación` y `04 Inteligencia Temporal`):

```
-- [01 KPIs Principales] Conciliación sobre órdenes efectivas (excluye devueltos y cancelados)
Venta Neta =
CALCULATE(
    SUM(stg_order_items[item_net_amount]),
    KEEPFILTERS(fct_orders[status] IN {"completed", "processing", "shipped"})
)

Costo de Ventas =
CALCULATE(
    SUM(stg_order_items[item_total_cost]),
    KEEPFILTERS(fct_orders[status] IN {"completed", "processing", "shipped"})
)

Utilidad Neta = [Venta Neta] - [Costo de Ventas]
% Margen Neto = DIVIDE([Utilidad Neta], [Venta Neta], BLANK())

Pedidos Efectivos =
CALCULATE(
    DISTINCTCOUNT(stg_order_items[order_id]),
    KEEPFILTERS(fct_orders[status] IN {"completed", "processing", "shipped"})
)

Ticket Promedio = DIVIDE([Venta Neta], [Pedidos Efectivos], BLANK())

-- [03 Clientes & 04 Inteligencia Temporal] Propagación controlada e indicadores dinámicos
Clientes Activos =
CALCULATE(
    DISTINCTCOUNT(fct_orders[customer_id]),
    CROSSFILTER(stg_order_items[order_id], fct_orders[order_id], Both),
    KEEPFILTERS(fct_orders[status] IN {"completed", "processing", "shipped"})
)

Venta Mes Anterior = CALCULATE([Venta Neta], DATEADD(Calendario[Fecha], -1, MONTH))
% Crecimiento MoM  = DIVIDE([Venta Neta] - [Venta Mes Anterior], [Venta Mes Anterior], BLANK())
```

06 Fase 5: Power BI DevOps (PBIP, Git/GitHub, CI/CD y Microsoft Fabric)

A diferencia de los flujos tradicionales basados en archivos binarios opacos (`.pbix`) que impiden auditar cambios línea por línea, este proyecto implementa una arquitectura moderna de **BI-as-Code / Power BI DevOps** en la carpeta `powerbi/` del repositorio:

Pilar DevOps	Implementación Técnica Real en el Proyecto	Valor en Entornos Productivos
1. Formato PBIP (`TMDL` + `PBIR`)	Archivo raíz `.pbip` desacoplado en: • **Modelo Semántico (`TMDL`):** código declarativo legible en `model.tmdl` , `relationships.tmdl` y `tables/*.tmdl` . • **Reporte (`PBIR`):** visuales estructurados en JSON y tema corporativo `Retail_Executive_Dark.json` .	Permite revisión de código (*Pull Requests*), trazabilidad exacta de cambios en fórmulas DAX y trabajo colaborativo sin conflictos binarios.
2. Control de Versiones (`Git / GitHub`)	Repositorio unificado que versiona simultáneamente los modelos SQL de **dbt** y el proyecto **PBIP**, gobernado por un `.gitignore` que bloquea cachés locales (`cache.abf` , `localSettings.json`) y llaves privadas cloud.	Historial inmutable de versiones, *rollbacks* seguros y cumplimiento estricto de seguridad de credenciales en la nube.
3. Automatización (`CI/CD GitHub Actions`)	Workflow automático `.github/workflows/bi_devops_ci.yml` + suite `scripts/validate_bi_devops.py` que se ejecuta en cada `push` y `pull_request` validando modelos dbt, 28 tests YAML, JSONs PBIR, modelo TMDL y KPIs DAX.	Detección temprana de errores sintácticos, rotura de esquemas o fugas accidentales de archivos sensibles antes de llegar a producción.
4. Ecosistema `Microsoft Fabric` & `Power BI Service`	Inclusión de manifiestos oficiales `.platform` bajo el esquema `fabric/gitIntegration/platformProperties/2.0.0` con `logicalId` persistentes, y despliegue interactivo final en **Power BI Service** (`app.powerbi.com`).	Mientras **Google BigQuery** opera como *Cloud Data Warehouse*, el artefacto BI es 100% compatible con sincronización nativa Git en Workspaces de Microsoft Fabric.

07 Fase 6: Dashboard Ejecutivo Final, Hallazgos y Recomendaciones

El entregable visual se diseñó bajo un sistema cromático oscuro de alto contraste (`#07151E`) pensado para lectura ejecutiva: el color **azul petróleo (`#0D83B8`)** guía las métricas operativas y comparativas, mientras el **acento ámbar dorado (`#F2C811`)** reserva la atención visual para la **Utilidad Neta** y su evolución mensual.



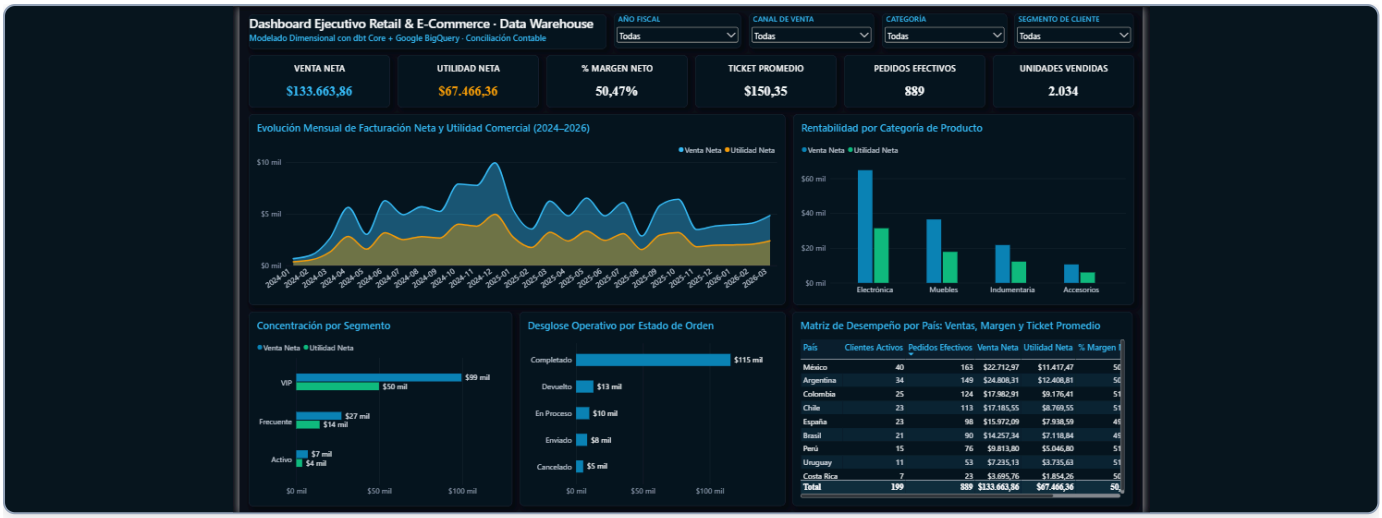


Figura 3. Dashboard Ejecutivo final publicado en Power BI Service conectado al modelo dimensional de Google BigQuery.

08 Interpretación Analítica de Resultados y Plan de Acción

8.1 Cinco Hallazgos Estratégicos del Negocio

1. **Crecimiento Saludable y Estacionalidad Q4 (2024–2026):** La serie mensual evidencia una aceleración sostenida en el segundo semestre de 2024 hasta alcanzar su pico histórico en **diciembre de 2024 (~`$10,000` mensuales)**. La curva de *Utilidad Neta* evoluciona en paralelo a la *Venta Neta* sin compresión de márgenes, confirmando que las promociones aplicadas no erosionaron la rentabilidad unitaria.
2. **Motores de Rentabilidad por Categoría: Electrónica** (`~$65 mil` en ventas | `~$32 mil` en utilidad) y **Muebles** (`~$37 mil` en ventas | `~$18 mil` en utilidad) concentran más del **75% del beneficio neto total** de la compañía, mientras *Indumentaria* y *Accesorios* cumplen un rol complementario de volumen y venta cruzada.
3. **Fuerte Concentración Pareto en el Segmento VIP:** Los **97 clientes VIP** (48.5% de la cartera activa, con compras acumuladas ≥ `$600`) generan **`$99,481.20` en Venta Neta (74.4% del total)** y **`$49,809.70` en Utilidad Neta**, registrando un ticket promedio de `$182.53` (+21% sobre el promedio general), seguidos por los segmentos *Frecuente* (`~$27 mil`) y *Activo* (`~$7 mil`).
4. **Diagnóstico de Fricción Operativa (Devoluciones vs. Cancelaciones):** Frente al bloque efectivo (*Completado* `~$115 mil`, *En Proceso* `~$10 mil` y *Enviado* `~$8 mil`), se pierden **`~$13 mil` en Devoluciones (`7.4%` de las órdenes)** y **`~$5 mil` en Cancelaciones (`3.7%`)**. Esto demuestra que el principal punto de fuga financiera ocurre en la post-venta/logística inversa y no durante el checkout.
5. **Liderazgo Regional Dual y Consistencia de Precios (9 Mercados): México** lidera en tracción operativa con `40` clientes activos y `163` pedidos efectivos (`$22,712.97`), mientras que **Argentina** lidera en ingresos netos (**`$24,808.31`**) y ganancia neta (**`$12,408.81`** en `149` pedidos), seguidos por **Colombia** (`$17,982.91`) y **Chile** (`$17,185.55`). El margen neto se mantiene altamente estable entre `49.7%` y `51.5%` en todos los mercados.

8.2 Recomendaciones Ejecutivas Accionables

1. Blindaje y Fidelización VIP

Implementar atención preferencial, envío bonificado y acceso anticipado en *Electrónica* y *Muebles* para los 97 clientes VIP que sostienen el **74.4%** de la facturación neta.

2. Escalamiento Frecuente → VIP

Activar campañas de *cross-selling* y paquetes promocionales (*bundles* de Electrónica + Accesorios) sobre los 72 clientes *Frecuentes* para elevar su ticket y superar el umbral de `$600`.

3. Auditoría de Logística Inversa

Inspeccionar causas de devolución por SKU y canal sobre los `~$13 mil` devueltos (`7.4%`). Reducir 2 puntos porcentuales esta tasa recupera margen neto directo sin costo de adquisición.

8.3 Guía Rápida de Reproducción Técnica End-to-End

```
# 1. Clonar el repositorio e instalar entorno analítico
git clone https://github.com/rubenbarrios-bigdata/retail-analytics-dbt.git
cd retail-analytics-dbt
python -m venv .venv && .venv\Scripts\activate
pip install dbt-core dbt-bigquery pyyaml

# 2. Poblar semillas, ejecutar transformaciones SQL y correr las 28 pruebas de calidad en BigQuery
dbt debug && dbt seed && dbt run && dbt test

# 3. Ejecutar la suite local de validación Power BI DevOps (PBIP + TMDL + Fabric Git Manifests)
python scripts/validate_bi_devops.py

# 4. Abrir 'powerbi/Dashboard dbt-BigQuery Retail Analytics.pbip' en Power BI Desktop
```
